ETX on MI300X: one day of measurement

Qwen3-30B-A3B bf16 decode, batch 1 · 2026-10-01 · 22 measured iterations

> The claim ETX exists to prove — that the kernels production actually runs, fused into one persistent kernel, beat the same kernels launched one at a time — was **false** at the start of this day and is **true** at the end of it. Fusion went from 0.235 ms behind a graph replay of the same tiles to 0.103 ms ahead. Against vLLM the system went from 21% behind to 3.7% behind. Of the eleven changes attempted, three paid; of the twenty-two investigations, eleven ended in a measured negative with an explanation, and those explanations are what moved the work.

1. Machine, protocol, and what may be compared

One box, one protocol, every number re-measured on it. **1× AMD Instinct MI300X VF** (SR-IOV virtual function), ROCm 7.14, 20 cores, 235 GB RAM. Qwen3-30B-A3B bf16, greedy, batch 1, 32 generated tokens checked token-for-token against HuggingFace greedy, plus `--check-dsl`: every DSL operator of the last layer within 2 bf16 ulps of a CPU reference. No performance number in this report comes from a run that failed either gate.

> **Two cautions that govern every comparison here.**
> *Across machines.* The 4.67 ms vLLM figure quoted in earlier reports is from a different, bare-metal machine with a different compiler; our megakernel builds to 15 register spills on this box against 9 there. Absolute milliseconds are not interchangeable between the two. Ratios and the fusion margin are, and both were confirmed on bare metal (§8).
> *Across timing bases.* vLLM's benchmark times host wall clock; ETX reports a device span. These differ by the host work ETX does per step, which was 0.197 ms when first measured. **Every headline number in this report is wall clock**, the basis vLLM is measured on. Device-span figures are labelled as such.

2. The result

Final configuration, same binary generation for all three rows, three interleaved runs each, all matching 32/32:

Configuration	Wall ms/token	Device ms/token	vs vLLM
vLLM 0.27.1, default	4.862	—	—
vLLM 0.27.1 + AITER	4.975	—	+2.3%
ETX megakernel	**5.042**	4.908	**+3.7%**
ETX unfused — same tiles, same plan, HIP-graph replay	5.145	5.107	+5.8%

Fusion margin: 0.103 ms (2.0%). The unfused baseline is deliberately the strong one. It is not eager launching; it captures the whole step into a HIP graph once and replays it, which is what AMD recommends and what vLLM itself does. The same tiles, the same task decomposition, the same XCD placement — the only variable is whether they run inside one persistent kernel or as graph nodes.

The day began at 5.895 ms device span with fusion *losing* by 0.235 ms. Measured on identical terms, ETX against ETX, the improvement is **−14.5%** and the fusion margin changed sign.

3. What paid, and what it was worth

Change	ms/token	Mechanism
`ETX_VL_COH` — data-class coherence	**−0.317**	Every task's acquire was `buffer_inv sc1`, invalidating the XCD's entire L2, so every phase began cold. DSL tiles now access activations with coherent sc0/sc1 operations and skip the acquire; weights stay cached; imported kernels keep their invalidate; a plain producer feeding a coherent consumer releases with a write-back.
Splitting a fold we had added	**−0.231**	The MoE top-k had been folded into all 448 expert tasks of a layer, each redoing a 128-logit softmax on wave 0 while fifteen waves waited at a barrier. Ablation priced the redundancy at 0.676 ms; restoring the standalone phase costs one hand-off and nets the difference.
Per-step host work made asynchronous	**−0.152**	Twenty synchronous HIP calls per step became stream-ordered asynchronous ones, a 241 KB memset of a disabled debug counter was removed, and a 50 μs poll sleep became a yield.
Non-temporal weight loads	**−0.106**	6.08 GB of weights are read once per token and have no reuse, yet they evicted what does: the MALL-resident KV cache and the activations. L2 hit rate 31.9% → 34.6%, DRAM read requests −3.9%.
Phase width 304 → 256 tasks	**−0.067**	Dense phases only; the MoE phases were already at their optimum width, which is why the effect is small.
Pruning unused kernel imports	**−0.015**	The megakernel's LDS arena is the maximum over every imported tile, used or not; `wvsplitk` (32768 B) was imported although the DSL GEMV replaces it. Arena 41448 → 29736 B.

4. What was measured and rejected

Eleven investigations ended in a negative. Each number is recorded in the source beside the code it concerns so that it is not retried blind.

Idea	Result	Why
Next-task argument prefetch	−0.003	Inside the spread. The argument path was not the exposed cost.
Three barriers per task reduced to two	**+0.095**	Keeping the next task's descriptor live across the body cost a tile-sized LDS allocation, 41448 → 61952 B. Even after moving its global load off the publication path it loses.
16-byte load path for `moe_down` (K=768)	**+0.144**	Correct, and slower: per-load address tracking and a second per-row wave reduction cost more than the wider loads save.
Attention block spread over 8 XCDs	+0.008	It is latency-bound, not parallelism-bound: `prep` moves 2.5 KB per task in a 5.9 μs body, 1.5% of one workgroup's streaming rate.
Batching the top-k tile's stores	+0.008	Its body is one write-through round trip that every coherent store sits behind, not 24 separable costs.
Skinny GEMM for the dense phases, B=2	**+0.29**	The second activation column cost 90% of the first — the tile is at the point where compute time equals stream time, so sharing the weight read buys little. A batched phase must also wait on every request's previous event, removing the slack replication provides.
HIP-graph capture of the per-step host sequence	0.000	Once the per-step calls are asynchronous there is nothing left for a graph to collapse.
Device-side per-step state reset	−0.008	Works: host reset 57.8 → 17.0 μs. But 14 μs returns as device time and the rest is absorbed by the wait. 0.16% does not earn a reset path, an exit counter, two memory fences and a fallback guard in every build.
Smaller workgroups for higher occupancy	not built	Waves/SIMD = min(512/VGPR, LDS-limited). At 128 VGPRs the register term is 4 whatever the workgroup size.
Fusing `prep` into `attn`	not built	Impossible as stated: `prep` reads and writes the same rows, so any shape with more than one task per KV head applies the norm and rope twice — wrong, not merely slow.
Blocking instead of polling for completion	−0.002	The 71 μs attributed to the wait strategy is launch-to-start and end-to-return latency.

5. Where the time goes

From a decode-range trace of the shipped build and `rocprofv3` counters, with the achievable rate for each phase's own bytes and width taken from a streaming benchmark at the tiles' 1024-thread width.

Component	ms/token	Owner
Streaming weights at the achievable rate	1.594	Hardware — irreducible at this precision
Workgroup start delay	0.414	Hardware dispatch; only 0.048 is ETX's, and ETX's spread is at or below the pure-streaming benchmark's own
Attention kernel's fixed cost	0.301	Imported vLLM kernel; it already runs *above* the HBM rate because the KV cache is MALL-resident
Hand-off between phases, 9 edges × 1.00 μs	0.433	ETX — near the event protocol's measured floor
Per-step host work	0.135	ETX
Our own DSL tiles	**1.42**	MoE bodies at ~70% of achievable (0.476), `prep` and `reduce` dependent chains of 8.5 and 8.1 round trips where 3–4 look necessary (0.548), the top-k tile's single write-through round trip (0.391)

ETX's orchestration is no longer the problem. Hand-off sits at the protocol floor, start delay is below the hardware's own dispatch jitter, the attention kernel is at the memory system's limit, and host work is 0.135 ms. The remaining gap to vLLM, 0.180 ms, is smaller than the tile-quality item alone — and kernel quality was never ETX's claim. What the compiler and runtime own is close to spent; what our hand-written tiles own is not.

6. Mechanism findings

The cost of an invalidating acquire, measured two ways

The redesign that flipped the fusion margin rests on one hardware behaviour, and it was measured on both machines with the same result. On the *bandwidth* side, a phase whose working set is L2-resident takes 0.76 μs without an invalidate and 3.14 μs with one — **4.1×**. On the *latency* side, never used in a design until now, a dependent load chain crossing `buffer_inv sc1` pays **1107 ns per hop against 103 ns — 10.6×** on bare metal and 11.4× on the VF. Reading activations coherently rather than invalidating and reading cached is 5.8× cheaper. For phases that are dependent chains, the latency figure is the governing one and it is larger.

Occupancy: half the machine's wave slots, and the limit is registers

Measured occupancy is **4.000 of 8 waves per SIMD**, identical minimum and maximum across all 1055 dispatches. Both resources bind at once: 512 VGPRs per SIMD divided by 128 per wave is 4, and before the import prune the LDS arena gave the same number. **85.9% of wave-cycles are spent waiting** while the memory unit reports zero stalls and LDS bank conflicts are negligible — too little memory-level parallelism, not too little bandwidth. Reaching 6 waves/SIMD needs VGPR $\leq$ 85 and LDS $\leq$ 21845 B simultaneously; that is a tile redesign, not a flag.

> **A hypothesis of ours that the counters refuted.** We expected the megakernel to be penalised because its register and LDS budgets are the maximum over all tiles, while separate launches each get their own. The unfused kernel measures VGPR 104 — also 4 waves/SIMD. Occupancy is not part of the fusion penalty. The counters did, however, state the fusion margin directly: mean occupancy per CU is **3.946 fused against 0.974 unfused**; the megakernel keeps 98.6% of compute units busy, separate launches about a quarter of that.

Instruction cache

The megakernel is one switch over roughly 400 task types, and the literature names instruction-cache thrashing as an unaddressed cost of this design. Measured here: **99.90% hit rate**, 6,958 misses against 6.8 million requests. Ruled out.

7. Two measurement errors we caught

Both were ours, both invalidated analysis we had already acted on, and both were found by checking rather than by a failure.

- **Traces taken during the prompt, not decode.** Three rounds of per-phase analysis used traces limited to the first eight steps, which are the teacher-forced prompt with sequence lengths of 1 to 8. vLLM's attention kernel partitions by a hardcoded 256 tokens and exits immediately past the sequence end, so sixteen of twenty attention tasks looked trivial and we concluded the phase had a 4.3× work imbalance. Re-traced in the decode range the spread is 1.17×; there is no imbalance. Timing was never affected — the benchmark only records steps after the prompt — but the attention block was understated by 0.21 ms/token.
- **A statistic that charged imbalance to skew.** The budget computed in-phase skew as a phase's span minus its *mean* body. A phase ends when its last task ends, so that quantity silently includes work imbalance. Recomputed properly, 0.414 ms is start delay and 0.244 ms is imbalance, and the router phase's apparent 4.34 μs of skew was 0.68.

A third error was found in the baseline itself: the streaming benchmark used 256-thread workgroups while the tiles use 1024, so the achievable-rate column was measured at a quarter of the memory parallelism. Corrected, the MoE residual falls from 0.532 to 0.476 ms/token and the router's from 0.164 to 0.046 — most of what we had charged to ETX there was the hardware floor.

8. Validation on bare metal

The same build, same gates, on the Hot Aisle bare-metal MI300X with ROCm 7.2.4 — the machine type every historical number in this project came from. The plan is byte-identical (60362 tasks, 436 events); only the register allocator differs (9 spills against 15).

Configuration	Device ms/token	2026-09-30 baseline
ETX megakernel	5.042	5.895
ETX unfused	5.266	5.660
vLLM default	4.887	4.836
Fusion margin	**+0.224**	**−0.235**

The claim ETX exists to prove now holds on the machine where it originally measured negative, and the improvement is **−14.5%** there as well. We are 3.1% behind vLLM on that box against 3.7% on the VF — both systems are faster on the VF and ours by less, so the relative position is slightly worse on bare metal. The mechanism findings transfer exactly: the invalidate penalty is 4.1× on both, and the streaming table agrees within a few percent on every row. Wall-clock figures were not captured there before the machine was released, so the bare-metal column is device-span only and is optimistic by an unmeasured amount of the same kind.

9. Batch behaviour, and a premise that was wrong

We expected a megakernel's advantage to grow with batch size, since a persistent kernel pays its scheduling cost once while launch-based systems pay per step. The measurement says the opposite.

Batch	vLLM ms/token	ETX ms/token	Ratio
1	4.861	5.267	1.08×
2	2.793	3.598	1.29×
4	1.820	3.466	**1.90×**
8	0.941	—	—

ETX does not batch; it replicates. Task counts are exactly linear in request count — 59,978 / 119,956 / 239,912 — because each additional request instantiates another copy of the whole task graph, so every request streams its own copy of the weights. vLLM reads a weight tile once and multiplies it by a B-column activation matrix, which is why its step time barely moves from batch 4 to 8 while its throughput nearly doubles. Interleaving does buy something — batch 4 costs 13.9 ms, not four times 5.27 — but that is latency hiding, not bandwidth sharing, and it cannot beat an approach whose weight traffic is constant in B.

The obvious remedy, a skinny GEMM serving B activation columns from one weight read, was built for the dense phases and **lost by 0.29 ms at B=2** for two reasons worth recording: the second column costs 90% of the first, because the tile sits at the point where compute time equals stream time and adding columns doubles the LDS traffic and arithmetic while saving only the weight stream; and a batched phase must wait on every request's previous event, removing the slack that replication provides. LDS also caps the design at B=2 — B=4 would need 82560 B against a 65536 B hardware limit. Batching ETX properly requires matrix instructions for B>1 and whole-layer batching with no per-phase join, including the MoE taking the union of the requests' experts. That is a separate project, not a switch.

10. What vLLM's own fusion is worth

To place our 2.0% on a scale, the same ladder measured for vLLM on this box:

vLLM configuration	Wall ms/token
Default — compiled, fused, full CUDA-graph replay	4.862
Without Inductor operator fusion	4.889
Compiled and fused, graphs disabled	32.710
Eager — no compilation, no graphs	35.134

vLLM's graph is worth **6.7×**; its operator fusion is worth 0.6%. Host dispatch dominates everything, and both systems have already removed essentially all of it — vLLM through graph capture, ETX through the persistent kernel. On top of that, ETX's megakernel adds 2.0% and vLLM's operator fusion adds 0.6%: the same order of magnitude, and both an order of magnitude below what avoiding per-operator dispatch is worth.

11. What remains

Item	ms/token	What it would take
MoE tile bodies at ~70% of achievable	0.476	Arithmetic has excluded bytes-in-flight and the serialised pass structure; the expert gather is 7% and the epilogue 5%. The remaining lead is that the dense tiles realise roughly a quarter of their waves' memory parallelism — unvalidated, and a four-phase correlation is not a mechanism.
`prep` and `reduce` dependent chains	0.548	8.5 and 8.1 round trips where 3–4 look necessary. Task boundaries offer no solution: fusing `prep` into `attn` is semantically impossible, and spreading it changes nothing. Shortening the chain inside the tile failed once on register pressure.
Top-k tile	0.391	At its floor under the coherence design: the body is one write-through round trip behind which every store sits. The only lever is not publishing coherently, which reintroduces the invalidate the design removed for four times more.
Register pressure → occupancy	unquantified	85.9% of wave-cycles wait with no memory-unit stalls, so more resident waves should hide latency. Needs VGPR 128 → 85 with LDS under 21845 B. The upper bound is unknown until built.
Hand-off, host work, start delay	0.98	At or below their floors; nothing cheap left.

The honest reading is that the compiler and runtime half of this project has reached its measured floor, and the remaining difference between ETX and vLLM at batch 1 is the quality of hand-written tiles against kernels AMD has tuned. That is worth saying plainly because it is not the thing ETX set out to prove — and because the thing it did set out to prove is now measured, on two machines, with the strong baseline rather than the weak one.

Qwen3-30B-A3B bf16, batch 1, greedy, 32 tokens verified against HuggingFace greedy and a CPU reference for every DSL operator of the last layer. Primary machine: 1× MI300X VF, ROCm 7.14. Validation: 1× MI300X bare metal, ROCm 7.2.4. vLLM 0.27.1. Measurements, logs, traces and profiler output are archived with checksums in `docs/DATA.md`; per-round findings in `results_2026-10-01/`; the per-phase budget in `docs/BUDGET-2026-10-01.md`.